



WORKSHOP • PYTHON

Python Além do Básico

Aplicações Reais do Mercado

De dados a backend: as bibliotecas, os frameworks e as boas práticas que aparecem no dia a dia do mercado.

O que vamos ver hoje

Agenda do encontro



Python no mercado

Onde dados e backend se encontram no dia a dia profissional



Django x FastAPI

Quando escolher cada framework web



Boas práticas

Estrutura de projeto, testes e versionamento



Bibliotecas por trás de tudo

As ferramentas mais usadas em cada frente



Bancos de dados

SQLite, PostgreSQL e a hora certa de migrar



Próximos passos

Como evoluir os estudos e o portfólio depois de hoje

Python no mercado hoje

Duas frentes, uma linguagem



DADOS

- Machine Learning
- Data Science
- Business Intelligence
- Análise exploratória de dados



BACKEND

- APIs e microserviços
- Automação de processos
- Integração entre sistemas
- Aplicações de alto volume

Hoje vamos aprofundar principalmente o lado Backend

Bibliotecas mais usadas em Dados

Machine Learning, Data Science e BI — visão geral



pandas

Manipulação e análise de dados tabulares
(DataFrames)



NumPy

Computação numérica e arrays multidimensionais



scikit-learn

Machine learning clássico: classificação, regressão,
clustering



TensorFlow / PyTorch

Deep learning e redes neurais em produção



Jupyter

Ambiente interativo para exploração e prototipagem



Matplotlib / Seaborn

Visualização de dados e geração de gráficos

Backend em produção: o que muda

Foco do workshop — não é só "fazer funcionar"



Alto volume de requisições

Milhares (ou milhões) de usuários acessando o mesmo sistema ao mesmo tempo.



Integração entre sistemas

APIs, filas de mensagens e webhooks conectando serviços diferentes.



Consistência e confiabilidade

Testes automatizados, tratamento de erros e monitoramento.



Padronização de código

Times grandes exigem convenções, arquitetura clara e documentação.

Django x FastAPI

Dois frameworks, duas filosofias



- Framework completo — "baterias inclusas"
- ORM, admin e autenticação prontos
- Ideal para aplicações completas e CRUDs robustos
- Curva de aprendizado um pouco maior



- Framework leve e assíncrono
- Tipagem com Pydantic + docs automática (Swagger)
- Ideal para APIs de alta performance e microserviços
- Mais liberdade (e responsabilidade) na arquitetura

Na prática: muitas empresas usam os dois — Django no backoffice, FastAPI nas APIs

Django: baterias inclusas

Framework full-stack desde 2005



MTV

Model · Template · View

- Arquitetura MTV (Model-Template-View)
- ORM próprio: models viram tabelas automaticamente, sem SQL manual
- Django Admin: painel administrativo completo gerado sozinho
- Sistema de autenticação e permissões já pronto
- Ótimo para: CRUDs, sistemas internos, backoffice e MVPs rápidos

FastAPI: performance e tipagem

Assíncrono por natureza, rápido por design



async / await
alta concorrência

- Construído sobre Starlette + Pydantic
- Suporte nativo a async/await para alta concorrência
- Validação de dados automática via type hints do Python
- Documentação Swagger/OpenAPI gerada sozinha, sem esforço extra
- Ótimo para: APIs REST, microsserviços e integrações de alta performance

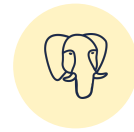
Bancos de dados: SQLite e além

Escolhendo a persistência certa para cada fase do projeto



SQLite

- Banco de dados embutido — um único arquivo
- Zero configuração: ótimo para protótipos e testes
- Não recomendado para alta concorrência



PostgreSQL / MySQL

- Servidor de banco dedicado
- Suporta múltiplas conexões simultâneas
- Escolha padrão em produção quando o projeto cresce

Regra prática: prototype com SQLite, migre para Postgres quando for para produção

Bibliotecas essenciais de Backend

O ferramental que aparece em quase todo projeto



requests

Consumir APIs externas via HTTP de forma simples



SQLAlchemy

ORM flexível para trabalhar com bancos relacionais



Pydantic

Validação e tipagem de dados (usado pelo FastAPI)



pytest

Testes automatizados — padrão de mercado em Python



Celery

Tarefas assíncronas e agendadas em fila de processamento



Docker

Empacotar e distribuir a aplicação de forma padronizada

Boas práticas e próximos passos

O que separa um script de um projeto de verdade

BOAS PRÁTICAS

- Estrutura de pastas clara (rotas, modelos, regras de negócio)
- Testes automatizados garantindo que nada quebre
- Variáveis de ambiente — nunca commitar senhas (.env)
- Versionamento com Git desde o primeiro commit
- README e documentação mínima do projeto

PRÓXIMOS PASSOS

- Publicar o projeto no GitHub com um bom README
- Documentar as decisões técnicas tomadas
- Explorar o ecossistema: Celery, Docker, CI/CD
- Contribuir com projetos open source em Python
- Praticar sempre — projeto pessoal vale mais que curso parado



Obrigado!

Perguntas? Bora trocar ideia.

Amanhã: mão na massa — vamos construir juntos, passo a passo.

[[seu contato](#) • [LinkedIn](#) • [GitHub](#)]